In [70]:
```
# import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

In [88]:
```
# 1. import database / data
conn = sqlite3.connect('customer_churn.db')
sql_query = """
        SELECT name
        FROM sqlite_master
        WHERE type='table'
   """
tables = pd.read_sql(sql_query, conn)

# create dataFrame for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn)
    globals()[f"df_{table_name}"] = df
    print(f"Create dataframe: df_{table_name}")

conn.close()
```

```
Create dataframe: df_db_customer
Create dataframe: df_db_subscription
Create dataframe: df_db_support
```

In [89]:
```
#find columns name
conn = sqlite3.connect('customer_churn.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    #Get column information
    columns_query  = f"PRAGMA table_info({table_name});" # PRAGMA query to help rea
    columns = pd.read_sql(columns_query, conn)
    print("columns :")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

```
Table Name: db_customer
columns :
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']

Table Name: db_subscription
columns :
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan
_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charge
s', 'cltv', 'churn_score']

Table Name: db_support
columns :
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

In [ ]:
```
# 2 Data cleaning
```

In [90]:
```
df_db_customer.head()
```

Out[90]:

	customerid	name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [85]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 4 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
dtypes: str(4)
memory usage: 804.0 bytes
```

In [81]:
```
# a. rename column - name
# b. drop columns - interest and pincode
# c. change data type -dob
# d. data standardzation - gernder
# e. fix missing value - country
```

In [91]:
```
# a. rename column - name
df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace=True)
df_db_customer
```

Out[91]:

	customerid	customer_name	country	state	gender	dob	interests	pincod
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	Non
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	Non
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	Non
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	Non
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	Non
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	Non
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	Non
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	Non
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	Non
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	Non
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	Non
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	Non
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	Non
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	Non

	customerid	customer_name	country	state	gender	dob	interests	pincod
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	Non
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	Non
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	Non
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	Non
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	Non
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	Non
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	Non

In [92]:
```
# b. drop columns - interest and pincode
df_db_customer.drop(df_db_customer.columns[-2:], axis=1, inplace= True)
```

In [93]:
```
df_db_customer
```

Out[93]:

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

In [94]:
```
# c. change data type -dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

In [15]:
```
df_db_customer
```

Out[15]:

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05
5	0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14
8	0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29
10	0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06

In [95]:
```
# d. data standardzation - gernder
df_db_customer['gender'].unique()
```

Out[95]:
```
<StringArray>
['Male', 'Female', 'Women', 'Men']
Length: 4, dtype: str
```

In [96]:
```
df_db_customer['gender'] =df_db_customer['gender'].replace({'Men' : 'Male', 'Women'
```

In [97]:
```
df_db_customer
```

Out[97]:

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05
5	0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
6	0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21
7	0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14
8	0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
9	0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29
10	0017-DINOC	shiva	India	Maharashtra	Male	1997-01-22
11	0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01
13	0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25
14	0019-GFNTW	Madhav	India	Uttar Pradesh	Male	1979-11-11
15	0020-INWCK	parvati	India	Delhi	Female	1986-02-28
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Male	1991-05-14
20	0023-UYUPN	sudevi	India	Maharashtra	Female	1977-10-06

In [98]:
```
# e. fix missing value - country

# how to find the value :
df_db_customer['country'].isna() # True mean : value nahi hai and False mean : valu
```

Out[98]:
```
0     False
1     False
2     False
3     False
4     False
5      True
6     False
7     False
8      True
9     False
10    False
11    False
12     True
13    False
14    False
15    False
16    False
17    False
18    False
19    False
20    False
Name: country, dtype: bool
```

In [99]:
```
# find data row of null value
df_db_customer[df_db_customer['country'].isna()]
```

Out[99]:

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

In [100…
```
# if you fill any vlaue so
df_db_customer['country'].fillna("Don't know")
```

Out[100…

```
0          India
1          India
2          India
3          India
4          India
5     Don't know
6          India
7          India
8     Don't know
9          Nepal
10         India
11         India
12    Don't know
13         India
14         India
15         India
16         India
17         India
18         India
19         India
20         India
Name: country, dtype: str
```

In [101…

```
df_db_customer[['country', 'state']]
```

Out[101…

	country	state
0	India	Maharashtra
1	India	Karnataka
2	India	Delhi
3	India	Nagaland
4	India	Delhi
5	NaN	Delhi
6	India	Meghalaya
7	India	Rajasthan
8	NaN	Kathmandu
9	Nepal	Kathmandu
10	India	Maharashtra
11	India	Karnataka
12	NaN	Telangana
13	India	Meghalaya
14	India	Uttar Pradesh
15	India	Delhi
16	India	Meghalaya
17	India	Rajasthan
18	India	Telangana
19	India	Uttar Pradesh
20	India	Maharashtra

In [102…

```
# create state and country map
state_conutry_mapping = df_db_customer.dropna(subset = ['country']).set_index('stat
state_conutry_mapping
```

Out[102…

```
{'Maharashtra': 'India',
 'Karnataka': 'India',
 'Delhi': 'India',
 'Nagaland': 'India',
 'Meghalaya': 'India',
 'Rajasthan': 'India',
 'Kathmandu': 'Nepal',
 'Uttar Pradesh': 'India',
 'Telangana': 'India'}
```

In [103…
```
# fill the value in to using state conntry mapping
df_db_customer['country'].fillna(df_db_customer['state'].map(state_conutry_mapping)
```

Out[103…
```
0     India
1     India
2     India
3     India
4     India
5     India
6     India
7     India
8     Nepal
9     Nepal
10    India
11    India
12    India
13    India
14    India
15    India
16    India
17    India
18    India
19    India
20    India
Name: country, dtype: str
```

In [104…
```
df_db_customer['country']= df_db_customer['country'].fillna(df_db_customer['state']
```

In [105…
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        21 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.1 KB
```

In [106…
```
df_db_subscription.head()
```

Out[106…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	M
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	A
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	M

In [107…

```
df_db_subscription.tail()
```

Out[107…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contra
16	0020-JDNXP	2022-01-19	Organic	2024-01-19	Premium	
17	0021-IKXGC	2021-07-07	Paid	2025-07-07	Standard	
18	0022-TCJCI	2023-09-14	Refferal	2024-09-14	Basic	M
19	0023-HGHWL	2020-06-23	Organic	2025-06-23	Premium	
20	0023-UYUPN	2022-12-31	Paid	2025-12-31	Standard	M

In [108…

```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     str
 1   subscription_start_date 21 non-null     str
 2   subscription_type       21 non-null     str
 3   renewal_date            21 non-null     str
 4   plan_type               21 non-null     str
 5   contract_type           21 non-null     str
 6   cancellation_date       6 non-null      str
 7   cancellation_reason     6 non-null      str
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

In [109…
```
# change here , columns Data type : str to datetime
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']
df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     str
 1   subscription_start_date  21 non-null     datetime64[us]
 2   subscription_type       21 non-null     str
 3   renewal_date            21 non-null     datetime64[us]
 4   plan_type               21 non-null     str
 5   contract_type           21 non-null     str
 6   cancellation_date       6 non-null      datetime64[us]
 7   cancellation_reason     6 non-null      str
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

In [ ]:
```
```

In [110…
```
df_db_support.head()
```

Out[110…

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

In [111…
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

In [115…
```
# change Column data type str to datetime
df_db_support['complaint_date']= pd.to_datetime(df_db_support['complaint_date'])
```

In [113…
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: datetime64[us](1), int64(1), object(1), str(3)
memory usage: 564.0+ bytes
```

In [114…
```
# here drop col_1 and Comment
df_db_support.drop(columns = ['col_1', 'comment'], inplace=True)
```

In [116…
```
df_db_support
```

Out[116…

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30
5	0017-IUDMW	2024-04-10	Y	25
6	0019-EFAEP	2024-09-27	Y	30
7	0022-TCJCI	2024-09-13	Y	10
8	0022-TCJCI	2024-09-14	N	90

In [65]:
```
# 3. Feature engineering and Data Analysis
```

In [117…
```
df_db_subscription.head(2)
```

Out[117…

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	A
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	A

In [118…
```
# Create a new column using existing col - churn flag
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_date']
```

In [119…
```
df_merge_data = df_db_subscription.merge(df_db_customer, on = 'customerid', how = '
```

In [120…
```
# very very Important
df_merge_data.shape # data not match
```

Out[120… (23, 20)

In [121…
```
df_db_subscription.shape
```

Out[121… (21, 12)

In [122…
```
df_db_subscription['customerid'].nunique() # finding Row nunique values
```

Out[122… 21

In [123…
```
df_db_customer['customerid'].nunique()
```

Out[123… 21

In [124…
```
df_db_support['customerid'].nunique()# find difference in customerid
```

Out[124… 7

In [125…
```
df_db_support['customerid'].size # Row size is 9 and nunique vlaue is 7
```

Out[125… 9

In [126…
```
# fix the customer id difference
df_db_support.head()
#finding multipal recode in customer id,
```

Out[126...

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30

In [127...

```
# add column numner of complaint and remove duplicate values
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customerid'
```

In [128...

```
df_db_support
```

Out[128...

	customerid	complaint_date	escalations	csat_score	complaint_count
0	0003-MKNFE	2024-08-28	N	60	2
1	0003-MKNFE	2024-08-28	Y	10	2
2	0013-EXCHZ	2024-01-20	Y	20	1
3	0013-MHZWF	2025-03-18	N	90	1
4	0013-SMEOE	2024-11-01	N	30	1
5	0017-IUDMW	2024-04-10	Y	25	1
6	0019-EFAEP	2024-09-27	Y	30	1
7	0022-TCJCI	2024-09-13	Y	10	2
8	0022-TCJCI	2024-09-14	N	90	2

In [129...

```
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('custom
```

In [130...

```
df_db_support
```

Out[130...

	customerid	complaint_date	escalations	csat_score	complaint_count
2	0013-EXCHZ	2024-01-20	Y	20	1
5	0017-IUDMW	2024-04-10	Y	25	1
1	0003-MKNFE	2024-08-28	Y	10	2
8	0022-TCJCI	2024-09-14	N	90	2
6	0019-EFAEP	2024-09-27	Y	30	1
4	0013-SMEOE	2024-11-01	N	30	1
3	0013-MHZWF	2025-03-18	N	90	1

In [131…
```
df_merge_data = df_db_subscription.merge(df_db_customer, on = 'customerid', how = '
```

In [132…
```
df_merge_data.shape
```

Out[132…
```
(21, 21)
```

In [133…
```
# Data Analysis
#1. Churn Rate

df_merge_data.to_csv('exported_churn_data.csv', index=False)
```

In [134…
```
df_merge_data.columns
```

Out[134…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

In [135…
```
chure_rate = df_merge_data['churn_flag'].mean()*100
print("Chure Rate = ", round(chure_rate,2), "%")
```

```
Chure Rate =  28.57 %
```

In [136…
```
# Retenion Rate
retenion_rate = 100 - chure_rate
print("Ratenio Rate = ", round(retenion_rate, 2), "%")
```

```
Ratenio Rate =  71.43 %
```

In [138…
```
# 3. churn by Plan type
churn_by_plan = (df_merge_data.groupby('plan_type')['churn_flag'].mean().mul(100).r
churn_by_plan
```

Out[138…

	plan_type	churn_rate_pct
0	Basic	60.00
1	Premium	14.29
2	Standard	22.22

In [139…
```
# 4 a. churn by state  + sum (revenue)& count of users -- home work **
# 4 b. churen by subscription type + sum (revenue)& count of users -- home work **
```

In [140…
```
churn_by_state = (df_merge_data.groupby('state')['churn_flag'].mean().mul(100).roun
churn_by_state

churn_by_sub_type = (df_merge_data.groupby('subscription_type')['churn_flag'].mean(
churn_by_sub_type
```

Out[140…

	subscription_type	churn_rate_pct
0	Organic	0.00
1	Paid	16.67
2	Refferal	83.33

In [141…

```
# 5 . ARPU - Avg Revenue Per User

arpu = df_merge_data['monthly_charges'].mean().round(2)
print('ARPU = ', round(arpu,2))
```

```
ARPU =  18.85
```

In [142…

```
# Calculate customer age
today  = pd.Timestamp.today()
customer_age = np.where(df_merge_data['dob'].notna(),
                        ((today - df_merge_data['dob']).dt.days/365.25).astype(int)
                        'Date of Birth not found'
                        )
# if want months :
# today = pd.Timestamp.today()

# customer_age_months = np.where(
#     df_merge_data['dob'].notna(),
#     ((today - df_merge_data['dob']).dt.days / 30.4375).astype(int),
#     'Date of Birth not found'
# )

# print("customer age in months:", customer_age_months)


print("customer age in years  :", customer_age)
```

```
customer age in years  : ['44' '30' '48' '25' '36' '37' '50' '27' '41' '32' '29' '4
5' '21' '34'
 '46' '40' '32' '26' '42' '35' '48']
```

In [143…

```
# 6. Avg customer Tenure
# Count of days users has used our service : cancellation date else current date
today  = pd.Timestamp.today()
df_merge_data['tenure_days'] = np.where(
            df_merge_data['cancellation_date'].notna(),
            (df_merge_data['cancellation_date'] - df_merge_data['subscription_start
            (today - df_merge_data['subscription_start_date']).dt.days
)

avg_tenure = df_merge_data['tenure_days'].mean()
print('Avg Tenure (Days)= ', round(avg_tenure))
```

```
Avg Tenure (Days)=  1549
```

In [144…

```
# 7. Revenue at risk   - revenue lost from churned users
revenue_at_risk = df_merge_data.loc[df_merge_data['churn_flag']==1 , 'monthly_charg
print("Revenue at risk(Rs 'K') =", revenue_at_risk)
```

```
Revenue at risk(Rs 'K') = 73.94
```

In [145…
```
# 8. Esclation Rate  :-
esclation_rate = (df_merge_data['escalations']=="Y").mean()*100
print("Escalation Rate = ", round(esclation_rate, 2), "%")
```

```
Escalation Rate =  19.05 %
```

In [146…
```
# 9. Avg Complaint  per User
avg_complaints = df_merge_data['complaint_count'].sum()/ df_merge_data['customerid'
print('Avg compliants per user = ', round(avg_complaints, 2))
```

```
Avg compliants per user =  0.43
```

In [151…
```
# 10. Correlation Esclation vs Churn
#df_merge_data['escalations'] = np.where(df_merge_data['escalations']=='Y',1,0) Thi
corr_df = df_merge_data[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df_merge_data['churn_flag'])
print("Correlation between esclation vs churn is =", round(correlation, 2))
```

```
Correlation between esclation vs churn is = 0.77
```

In [152…
```
# 11 .chure risk - create a column using Existing col
conditions = [
    (df_merge_data['churn_score']<50),
    (df_merge_data['churn_score']>=50) & (df_merge_data['churn_score']<70),
    (df_merge_data['churn_score']>=70)
]
choices = ['low','med', 'high']
df_merge_data['churn_risk'] = np.select(conditions, choices, default='unkown')
```

In [153…
```
df_merge_data[['churn_risk', 'churn_score']].tail()
```

Out[153…

	churn_risk	churn_score
16	med	62
17	low	27
18	high	99
19	low	7
20	low	47

In [ ]:
```
```

In [154…
```
# 4. Visualization using Matplotlib
df_visual = df_merge_data.copy()
```

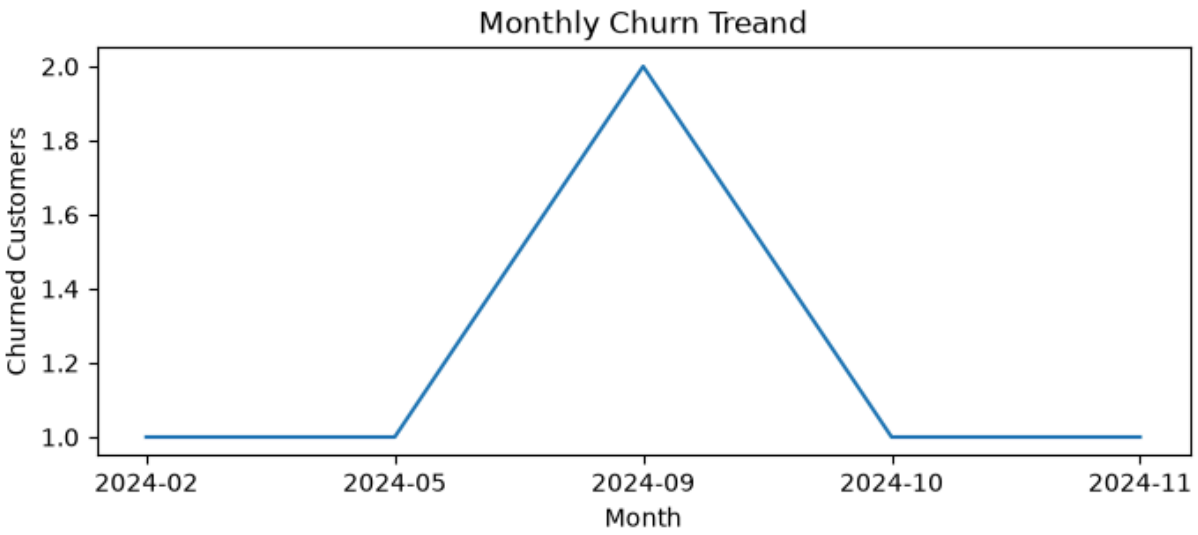
In [161…
```
df_visual.columns
```

Out[161…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [166…
```
# 4.1 monthly churn Trend(Time series KPT)
df_visual['cancellation_month']= df_visual['cancellation_date'].dt.to_period('M')

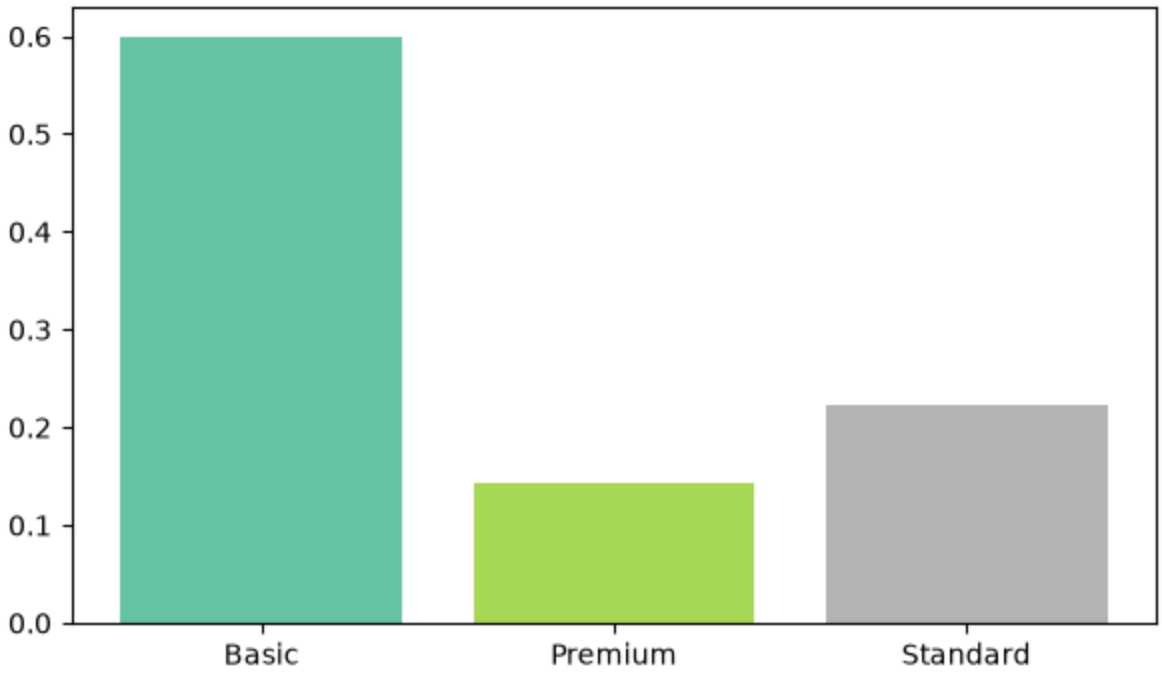
churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_month')
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values)
plt.title('Monthly Churn Treand')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show
```

Out[166…
```
<function matplotlib.pyplot.show(close=None, block=None)>
```



In [174…
```
# 4.2 churn by plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()
# colors= ['yellow', 'purple','blue']
colors  = plt.cm.Set2(np.linspace(0,1 ,len(churn_plan)))
plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color= colors )
```

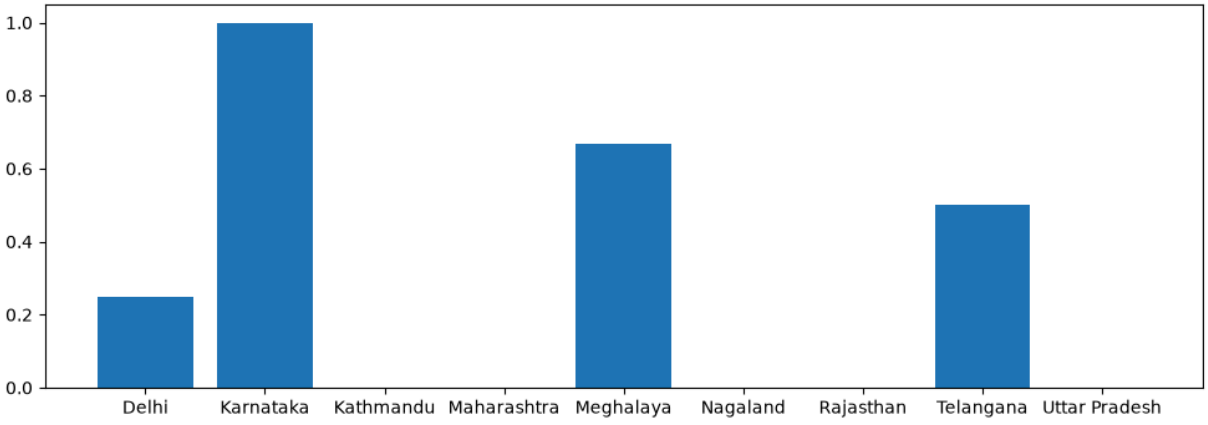
Out[174…
```
<BarContainer object of 3 artists>
```



In [175…
```
# 4.3 churn  by states
churn_state = df_visual.groupby('state')['churn_flag'].mean()

plt.figure(figsize=(12,4))
plt.bar(churn_state.index, churn_state.values,)
```

Out[175…
```
<BarContainer object of 9 artists>
```



In [ ]:
```
```

In [176…
```
# Visulaization using Seaborn
```

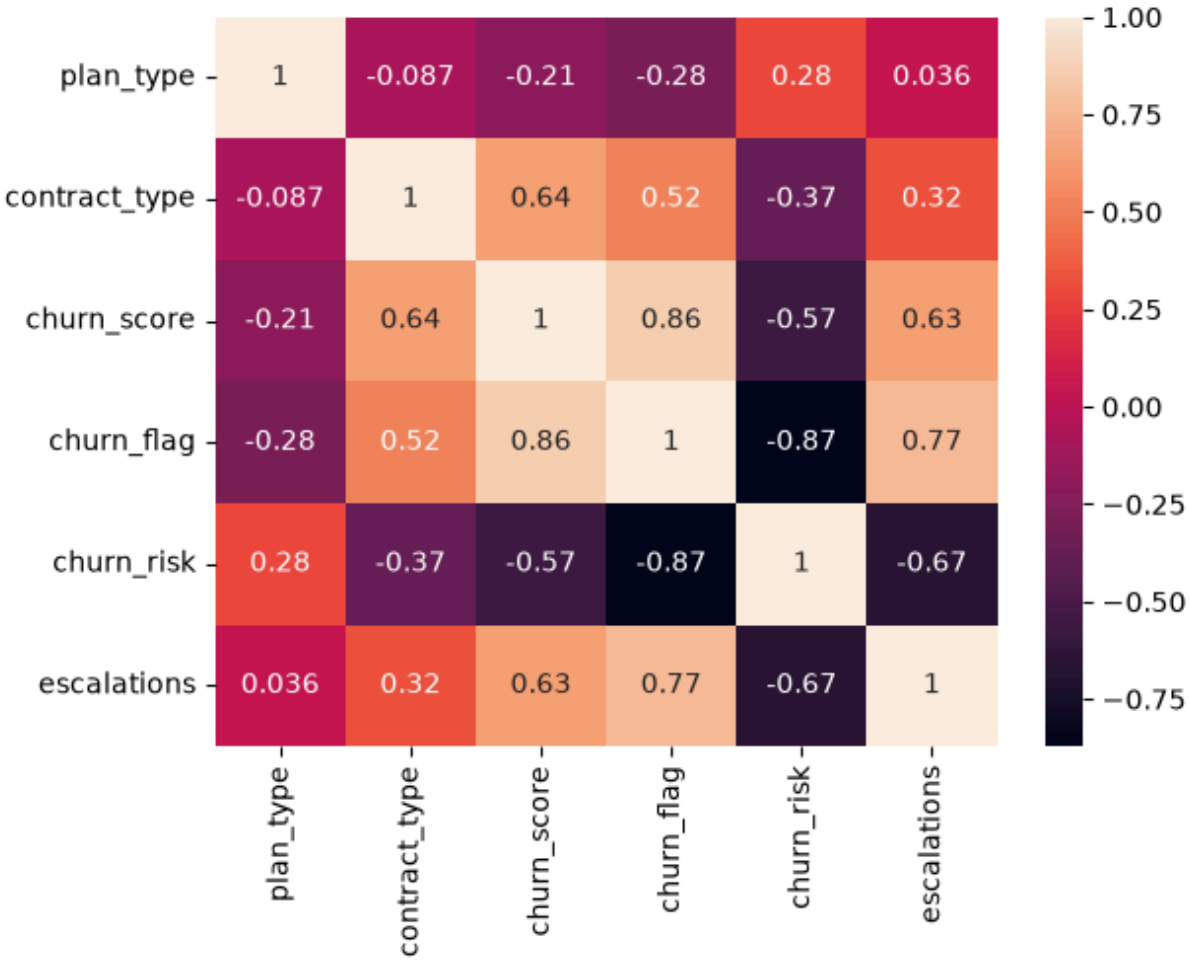
In [178…
```
# encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

Out[178…
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

In [179…
```
# incorrect method of Encoding -as number are not assigned based on priority
df_encoded = df_visual[[ 'plan_type', 'contract_type',  'churn_score',
       'churn_flag','churn_risk', 'escalations']]
categorial_cols = ['plan_type', 'contract_type','churn_risk']
for col in categorial_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

In [181…
```
# heatmap (correlation matrix) Heatmap only we use numeric type of vlaus
sns.heatmap(df_encoded.corr(), annot = True)
```

Out[181…
```
<Axes: >
```



In [191…
```
df_visual['plan_type'].unique()
```

Out[191…
```
<StringArray>
['Standard', 'Premium', 'Basic']
Length: 3, dtype: str
```

In [192…
```
# Correct method of Encoding -based on priority
df_encoded = df_visual[['plan_type','contract_type','churn_score','churn_flag','chu
order_cols = {
    'plan_type' : ['Basic','Standard','Premium'],
    'contract_type' : ['Monthly','Annual'],
    'churn_risk' : ['low','med','high']
    }
for col, order in order_cols.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categories
```

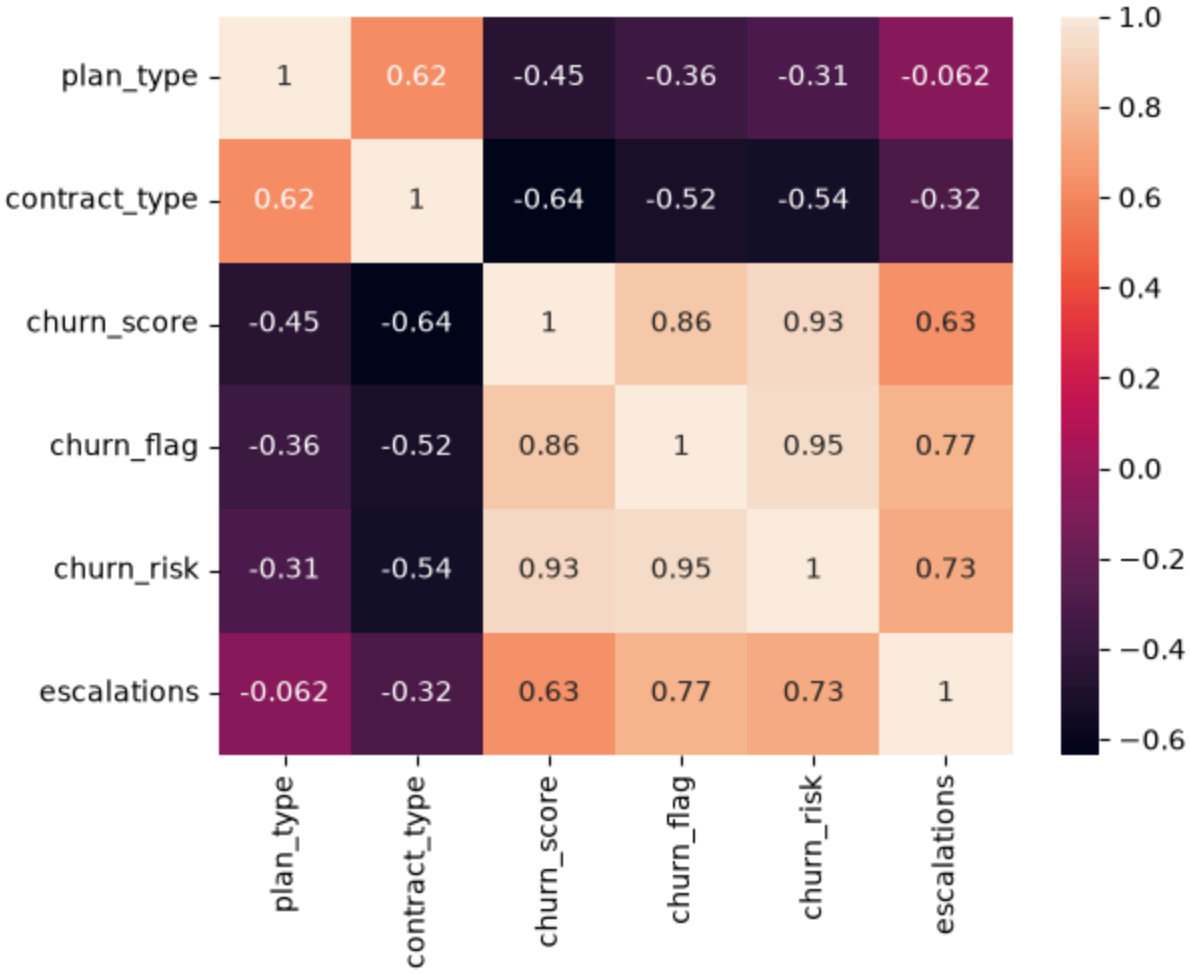
In [193…
```
df_encoded.head()
```

Out[193…

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	1	1	12	0	0	0
1	2	1	91	1	2	1
2	0	0	34	0	0	0
3	2	1	8	0	0	0
4	1	0	88	1	2	1

In [194…
```
# heatmap (correlation matrix) Heatmap only we use numeric type of vlaus
sns.heatmap(df_encoded.corr(), annot = True)
```

Out[194… <Axes: >

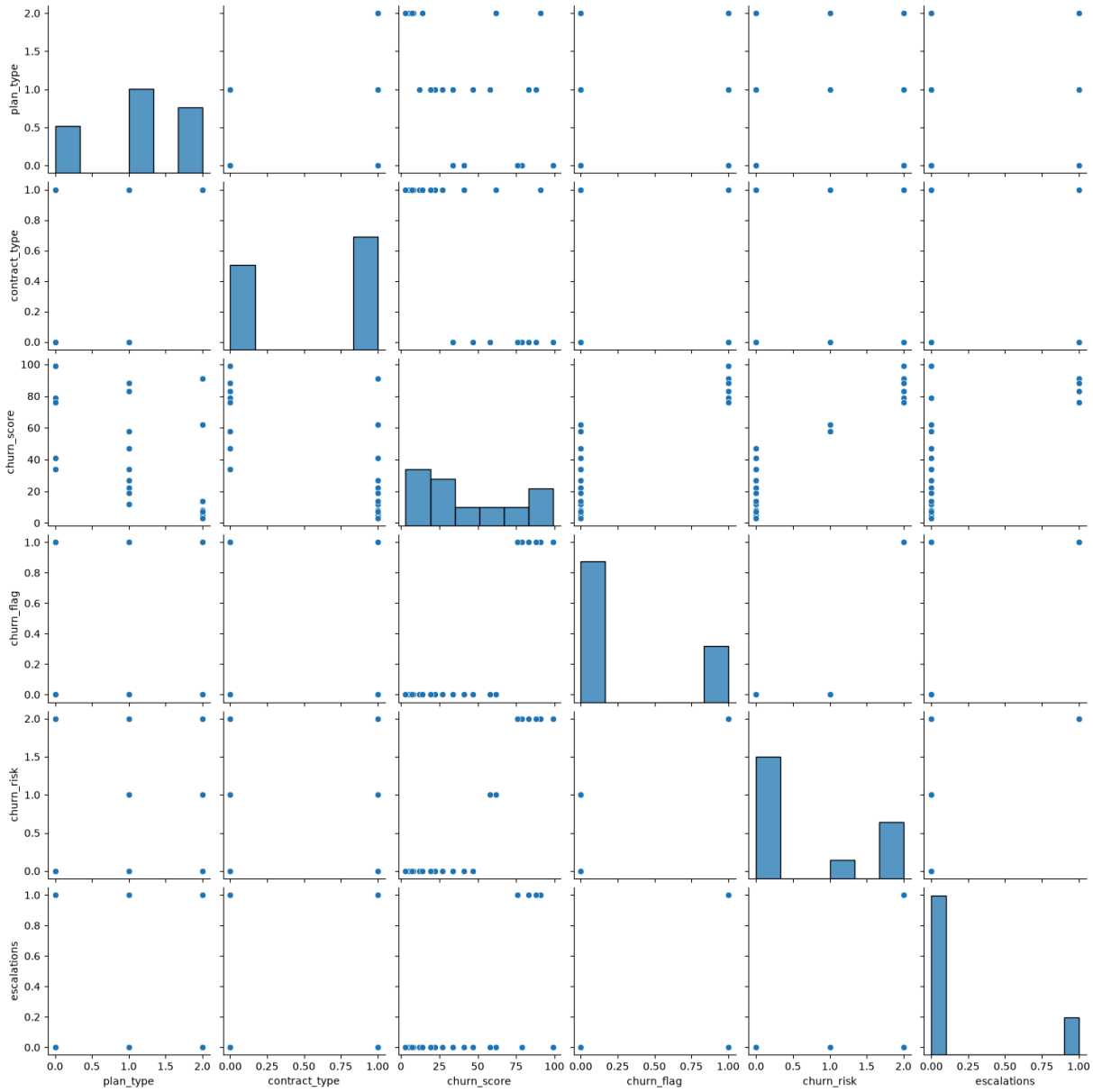


In [195…

```
# Pairplot - relationship in a data set
sns.pairplot(df_encoded)
```

Out[195…

```
<seaborn.axisgrid.PairGrid at 0x1df9d55fb60>
```

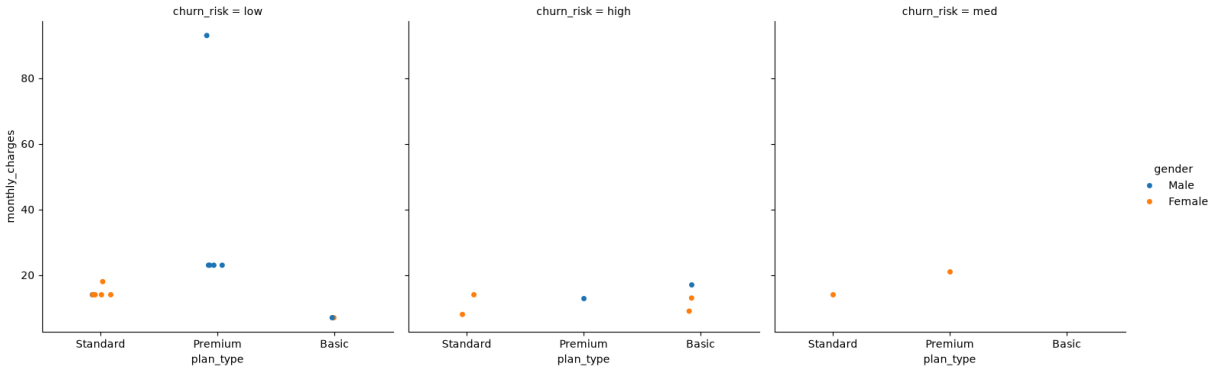


In [197…

```
# catplt/Facegrid plot - multi-dim comparison
sns.catplot( data = df_visual,
    x = 'plan_type',
    y = 'monthly_charges',
    hue = 'gender',
    col = 'churn_risk' )
```

Out[197…

```
<seaborn.axisgrid.FacetGrid at 0x1df9d55f8c0>
```



In [199…

```
# Pivot Table
pd.pivot_table(
    data = df_visual,
    values = 'churn_flag',
    index = 'plan_type',
    aggfunc = 'mean'
).reset_index()
```

Out[199…

	plan_type	churn_flag
0	Basic	0.600000
1	Premium	0.142857
2	Standard	0.222222

In [201…

```
pd.pivot_table(
    data = df_visual,
    index = 'plan_type',
    values = ['monthly_charges','customerid', 'churn_flag'],
    aggfunc = {
        'monthly_charges' : 'sum',
        'customerid' : 'nunique',
        'churn_flag': 'mean'
    }
)
```

Out[201…

plan_type	churn_flag	customerid	monthly_charges
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

In [ ]:

In [ ]:

In [ ]: